

오늘의 작업 기록 — 2026년 6월 21일

mini-ledger **중분 2 (Bank) 착수** — `field` + `createAccount` 완성 **오늘의 성격**: 코드 양은 적지만 개념 *깊이*가 깊은 날. 객체 생성·`this`·생성자 vs 메서드·타입 안전성을 *질문으로 끝까지 파고든* 날. 환경: 아이폰 + Termius(SSH) + VPS (*hostname*을 `ledger` 로 변경) / nano · javac · git · gh

한눈에 — 오늘 한 5가지

- VPS 정체성 정리 — hostname `8k-paper-key` → `ledger`
- 중분 2 착수 — Bank의 `field` + `createAccount` 작성
- ★깊은 개념 학습★ — 객체 생성(`new`), `this` , 생성자 vs 메서드, 타입 안전성
- 디버깅 케이스 — 작업 폴더(working directory) 이슈 진단·해결
- 커밋·푸시 — 잔디 (commit `e02b204`)

어제(6/20)가 "읽기 → 쓰기"의 첫 발이었다면, 오늘(6/21)은 "왜 이렇게 쓰는가"를 끝까지 캐물은 날이다.

Phase 0 — VPS 정체성 정리 (hostname 변경)

왜: 프롬프트의 `8k-paper-key` 는 옛 8-K 트레이딩 봇 때 droplet에 붙인 이름. 백엔드 환경에 안 맞아서 교체.

프롬프트 구조 이해:

```
root @ 8k-paper-key : ~ #
|           |           |
사용자   hostname   현재폴더 root표시
```

명령:

```
hostnamectl set-hostname ledger
sed -i 's/8k-paper-key/ledger/g' /etc/hosts
exec bash
```

- 1번 → 머신 이름 변경
- 2번 → `/etc/hosts` 의 옛 이름도 교체 (안 하면 "unable to resolve host" 경고)
- 3번 → 셸 새로고침 → 프롬프트 `root@ledger:~#`

왜 `Ledger` 인가 (= 원장, 元帳): 모든 금전 거래(입금·출금·이체)와 잔액이 기록되는 장부. 내가 만드는 mini-ledger가 말 그대로 *계좌 원장*이고, 핀테크의 핵심 개념(은행·결제·블록체인=분산원장). 머신 이름 하나가 *내가 만드는 것 + 들어갈 분야*를 담음.

배운 것: hostname의 정체, 프롬프트 읽는 법(이게 Phase 3 디버깅의 복선이 됨).

Phase 1 — 중분 2 접근법 (스케폴딩 한 칸 줄임)

- 어제는 Account *스켈레톤*을 짰지만, **오늘 Bank는 스켈레톤 없이** 스펙만 보고 작성 (자립 궤적).
- 새 도구 2개:
- Map** — 이름표(key)로 물건(value)을 꺼내는 *사물함/전화번호부*. Bank에선 "계좌 id → Account 객체".
- 제네릭** — `Map<String, Account>` 의 `<String, Account>` = "String 키에 Account 값만 담는다"는 *타입 약속*. 잘못된 타입 넣으면 *컴파일 단계*에서 잡힘.
- 도구가 달랐다: 어제 `cat heredoc` 은 *완성본을 붙여넣는 특수 케이스*. 직접 작성할 땐 **nano로 쓰고 → javac → 고치고 반복**이 정석.

Phase 2 — createAccount 작성 + 깊은 개념 ★오늘의 핵심★

2-1. 첫 시도 — 내가 짠 코드 (실수 3개)

```
public void createAccount(String id, String owner, long initialBalance) {
    this.id = id;                // x 실수 1
    this.owner = owner;          // x
    this.balance = initialBalance; // x
    Account account = new Account(id, owner, initialBalance) // x 실수 2: 세미콜론 없음
    accounts.put("김철수", "010-1111-1111");                // x 실수 3: 예시값 그대로
}
```

#	실수	왜 틀렸나
1	<code>this.id/owner/balance = ...</code>	어제 <i>Account</i> 생성자가 하던 일을 복붙. 근데 Bank엔 id/owner/balance 필드가 없다 → 컴파일 에러. (아래 깊은 질문에서 정리)
2	<code>new Account(...)</code> 뒤 <code>;</code> 누락	문장 끝엔 세미콜론
3	<code>put("김철수", "010-...")</code>	과제의 <i>예시값을 그대로</i> 넣음. 내 데이터(<code>id</code> , <code>account</code>)여야 함 + 타입도 틀림

2-2. 깊은 질문 — "왜 Bank엔 `this.id = id` 가 없나?" ★오늘의 진짜 수확★

내가 끝까지 물고 늘어진 질문. 답을 4단계로 정리:

- (1) "건너뚱" 게 아니라 "호출한" 거다. `new Account(id, owner, initialBalance)` 이 한 줄이 → **Account의 생성자를 실행**한다. 그 생성자가 바로 `this.id = id; this.owner = owner; this.balance = initialBalance;` 묶음. → 그 줄들을 *생략한 게 아니라, (Account 버전을) 불러서 쓴 것*. Bank에서 다시 안 쓰는 이유 = 이미 Account가 갖고 있고 `new` 가 자동 실행하니까.
- (2) `this` 가 가리키는 대상이 다르다. `this` = "지금 이 코드가 속한 객체". - Account 생성자 안 `this` = *만들어지는 Account* → `this.id` = Account의 id (Account엔 id 필드 있음✓) - Bank 메서드 안 `this` = *Bank 자신* → `this.id` = Bank의 id인데, **Bank엔 id 필드가 없다** (accounts 맵뿐) → 에러 ✕
- (3) **역할이 다르다.** - **Account** 생성자 = 계좌 *하나를 만들며 그 계좌의 속성을 채움*. - **Bank.createAccount** = 계좌를 *만들어서(생성자 호출) 보관함*. - createAccount가 id/owner/balance를 받는 이유 = *자기가 쓰려는 게 아니라 new Account(...) 에 넘겨주려고*** (전달자 역할).
- (4) **비유 — 봉어빵:** - **Account** = 봉어빵 + *틀에 팔 넣는 공정*(생성자) - **Bank** = 봉어빵 *진열 매대*. 매대는 팔이 *없다*(봉어빵이 아니니까), *칸(map)*만 있음. - `new Account(...)` = "틀로 봉어빵 굽기"(팔 들어감) / `accounts.put(id, account)` = "구운 봉어빵을 id 칸에 진열" - *"왜 매대가 팔을 안 넣어?"* → 그건 *굽는 공정*(생성자)의 일이지 *매대(Bank)*의 일이 아니다.

2-3. 또 하나의 함정 — 예시값 vs 내 변수

`"김철수", "010-1111"` 은 과제가 `put` 문법을 보여준 예시값이지 "이걸 넣어라"가 아니었다. - 예시 = *모양(문법)을 가르침* → `put(키, 값)` 패턴 - *내용은 내 데이터로 교체* → 키 = `id` , 값 = `account` - + **타입 안전성**: map이 `Map<String, Account>` 라 값은 *Account여야*한다. 전화번호 문자열 넣으면 컴파일 에러 → 이게 *제네릭의 안전장치*다. - (어제 "설명 ≠ 코드"의 사촌: *예시값 ≠ 내 변수*.)

2-4. 완성된 createAccount (2줄)

```
public void createAccount(String id, String owner, long initialBalance) {
    Account account = new Account(id, owner, initialBalance); // 설계도로 계좌 객체 찍기
    accounts.put(id, account);                                // id 칸에 보관
}
```

- 1줄: `new Account(...)` → Account 생성자 실행 → 새 계좌 객체 생성 (속성은 생성자가 채움)
- 2줄: `accounts.put(키=id, 값=account)` → 맵에 보관

2-5. 완성된 Bank.java (전체)

```
import java.util.Map;
import java.util.HashMap;

public class Bank {
    private Map<String, Account> accounts = new HashMap<>();

    public void createAccount(String id, String owner, long initialBalance) {
        Account account = new Account(id, owner, initialBalance);
        accounts.put(id, account);
    }
}
```

- `import` 두 줄: Map·HashMap 가져오기 (파일 맨 위)
- `private Map<String, Account> accounts` = Bank의 *유일한 필드* (계좌 보관소)
- 중복 id 체크(`containsKey`)는 *오늘은 생략*(기본 버전), 다음에.

Phase 3 — 디버깅 케이스: 작업 폴더(working directory) 이슈

오늘 가장 큰 *디버깅 수업*.

증상:

```
Bank.java:5: error: cannot find symbol
symbol: class Account
...
fatal: not a git repository (or any of the parent directories): .git
```

진단 — 프롬프트가 단서: `root@ledger:~#` 의 `~` 는 **홈 폴더**. *mini-ledger 안이 아니었다*.

근본 원인: Phase 0에서 `exec bash` 하면서 *홈으로 돌아왔는데* `cd ~/mini-ledger` 를 안 함 → `cat > Bank.java` 가 **홈(/root)**에 파일을 만듦. 그런데 Account.java와

git repo는 **mini-ledger** 안에 있음. 따로 놓. - `cannot find symbol: class Account` → javac가 *같은 폴더*에서 Account.java를 못 찾음 (다른 폴더라서) - `fatal: not a git repository` → 홈은 git repo가 아님 (repo는 mini-ledger에 있음)

해결:

```
mv ~/Bank.java ~/mini-ledger/ # 홈의 Bank.java를 mini-ledger로 이동
cd ~/mini-ledger # 프로젝트 폴더로 진입
javac *.java # 이제 같은 폴더에 Account.java 있음 → 컴파일 OK
git add . && git commit -m "add Bank with createAccount" && git push
```

교훈: - 프롬프트로 *내 위치* 확인: `~` (홈) vs `~/mini-ledger` (프로젝트) - **작업 전** `cd ~/mini-ledger` 습관 — 코드·git이 다 거기 있음 - `javac` 는 *같은 폴더의 다른 클래스* 를 찾는다 → 서로 참조하는 파일은 *같은 폴더*에 뒤야 함

Phase 4 — 커밋 (git: 새 프로젝트 vs 매일 작업)

어제 vs 오늘 명령이 다른 이유:

	어제 (새 프로젝트 시작)	오늘 (기존에 추가)
<code>git init</code>	✓ 1회성	✗ 이미 됨
<code>.gitignore</code> 생성	✓ 1회성	✗ 이미 있음
<code>gh repo create</code>	✓ 1회성 (생성+연결+첫push)	✗ 이미 있음
<code>git add/commit/push</code>	✓	✓ 매일

- 새 프로젝트 = `init` + `gh repo create` 포함 / **매일 작업** = `add.commit.push` 만
- `git push` 에 `origin main` 안 붙여도 됨 → 어제 `--push` 가 추적 연결(`-u`)을 해놔서
- 결과: commit `e02b204` , push `9e671d6..e02b204 main -> main`

오늘의 산출물

- ✓ **Bank 클래스** — `field` (Map) + `createAccount` (object 생성 + map 보관)
- ✓ 커밋 `e02b204` + push → 잔디
- 🔗 github.com/Taewoo-Won/mini-ledger (Account.java + Bank.java)
- 📁 VPS hostname → `ledger`

다음 작업 — 중분 2 남은 것

- `getAccount` — `accounts.get(id)` + 없으면 throw (어제 패턴)
- **transfer** — *오늘의 진짜 문제*. "검증 먼저"를 두 계좌로 확장. 출금 성공 후 입금 실패 시 **돈 중발**방지가 핵심. (= 트랜잭션/ACID 사고의 출발점)
- 읽을 것: 점프투자바 03-08 맵(85-89), 03-07 리스트(75-84, 제네릭)

오늘 배운 핵심 개념 (한 줄 사전) — 확장판

개념	한 줄
객체 생성 <code>new</code>	클래스(설계도)로 <i>실제 객체</i> 를 찍어냄. <code>new Account(...)</code> → Account 생성자 실행
클래스 vs 객체	설계도 / 그 설계도로 찍은 실체
생성자 vs 메서드	생성자=객체 태어날 때 자기 필드 채움 / 메서드=객체가 하는 일
<code>this</code>	"지금 이 코드가 속한 객체". 문맥마다 가리키는 대상이 다름
필드 위임	Bank가 <code>this.id</code> 안 함 → <code>new Account(...)</code> 가 Account 생성자에 위임
전달자(relay)	<code>createAccount</code> 는 받은 값을 자기가 안 쓰고 <code>new Account(...)</code> 에 넘김
Map / HashMap	key→value 사물함. <code>put</code> (넣기)· <code>get</code> (꺼내기)· <code>containsKey</code> (있나)
제네릭 <code><String, Account></code>	타입 약속. 엉뚱한 타입은 <i>컴파일 단계</i> 에서 차단
타입 안전성	<code>Map<String, Account></code> 에 String 값 넣으면 컴파일 에러
예시값 ≠ 네 변수	예제는 <i>문법 모양</i> 을 가르침, 내용은 내 데이터로 교체
working directory	현재 폴더. 프롬프트 <code>~</code> (홈) vs <code>~/mini-ledger</code> 로 확인
<code>javac</code> 클래스 탐색	같은 폴더의 다른 클래스를 찾음 → 참조 파일은 같은 폴더에
hostname	머신 이름. <code>hostnamectl set-hostname</code>
git: 1회성 vs 매일	<code>init</code> / <code>gh repo create</code> =한 번, <code>add.commit.push</code> =매일

오늘의 비유 모음

- **붕어빵/매대** — Account(붕어빵+굽는 공정=생성자) vs Bank(진열 매대=계좌 보관소). 매대는 팔을 안 넣는다.
 - **사물함/전화번호부** — Map은 이름표(key)로 물건(value)을 꺼내는 보관함.
 - **설계도/실체** — 클래스는 설계도, `new` 는 그 설계도로 찍어낸 실제 객체.
-

한 줄 회고

오늘 코드는 2줄(createAccount)이지만, 그 2줄을 *완전히 이해하려고* "왜 Bank엔 `this.id` 가 없나"를 끝까지 캐물었다. 그게 *OOP의 핵심*(클래스 vs 객체, 생성자 vs 메서드, 위임)이고 — 베껴 넘겼으면 못 배웠을 걸 *질문해서 내 것으로* 만들었다. 거기에 *작업 폴더 디버깅*까지 직접 풀었다. **양보다 깊이의 날.** 어제가 "쓰기 시작", 오늘은 "왜 그렇게 쓰는지 이해". 이게 진짜 실력으로 쌓인다.